



RAPPORT DE PROJET DJANGO

BookNest - Bibliothèque personnelle en ligne



2025-2026

TRACY FRANCK MUKENDI

<https://github.com/Tranck04/django-booknest-tracy-franck-mukendi>

Table de matière

1. Introduction	3
1.1 Contexte	3
1.2 Problème traité	3
1.3 Objectifs du projet	3
1.4 Conformité au cahier des charges	3
2. Analyse et conception	5
2.1 Acteurs	5
2.2 Cas d'utilisation principaux	5
2.3 Modèle de données	5
Category	5
Book	6
UserBook	6
2.4 Schéma relationnel	6
2.5 Logique anti-doublon	8
3. Réalisation technique	9
3.1 Architecture générale	9
Arborescence simplifiée	9
3.2 Applications et configuration	9
Extrait de configuration	10
3.3 Modèles, migrations et administration	10
3.4 URLs et vues	10
Routes publiques	10
Routes privées	10
3.5 Templates et interface utilisateur	11
3.6 Formulaires, CRUD et anti-doublon	11
3.7 Gestion des couvertures de livres	12
3.8 Gestion des mots de passe	12
3.9 Import du catalogue (271 379 livres)	12
3.10 Interface utilisateur et navigation	14
3.11 Authentification et sécurité	14
3.8 Suivi personnel, avis et statistiques	14
4. Tests et validation	15
4.1 Stratégie de test	15
Commande d'exécution	15
4.2 Résultat d'exécution	15
4.3 Couverture fonctionnelle	15
4.4 Erreurs rencontrées et corrections	15
4.5 Points de contrôle avant remise	16

5. Conclusion	17
5.1 Bilan	17
5.2 Compétences mises en pratique	17
5.3 Limites	17
5.4 Pistes d'évolution	17
6. Annexes	18
A. Instructions d'installation	18
B. Dépendances principales	18
C. Dépôt GitHub	18
D. Liste indicative des commits	Error! Bookmark not defined.

1. Introduction

1.1 Contexte

Dans le cadre du cours Django, il a été demandé de réaliser une application web complète respectant l'architecture MVT de Django. Le projet devait mettre en œuvre les notions essentielles du cours : environnement virtuel, applications Django, modèles, migrations, administration, vues, templates, formulaires, authentification, opérations CRUD, tests automatisés et documentation.

Le projet choisi est BookNest, une bibliothèque personnelle en ligne. L'application permet de consulter un catalogue de 271 379 livres issus du fichier books.csv, d'organiser les ouvrages par catégories, de gérer une bibliothèque propre à chaque utilisateur et de suivre sa progression de lecture. Une commande d'import dédiée (import_books) avec traitement par lots (bulk_create) et gestion des conflits (ignore_conflicts) a permis d'importer l'intégralité du dataset.

1.2 Problème traité

Un lecteur peut rapidement perdre le suivi de ses livres : ouvrages déjà lus, livres en cours, livres à lire plus tard, notes personnelles ou avis. Une simple liste ne suffit pas lorsque l'on souhaite aussi classer les livres par catégorie, éviter les doublons et conserver des statistiques de lecture.

BookNest répond à ce besoin avec un catalogue commun et des bibliothèques personnelles rattachées aux comptes utilisateurs. Un même livre n'est stocké qu'une seule fois dans le catalogue, puis il est relié aux utilisateurs qui l'ajoutent à leur collection.

1.3 Objectifs du projet

Les objectifs fonctionnels principaux sont les suivants :

- afficher une page d'accueil, une page à propos, un catalogue de livres et une fiche détaillée par ouvrage ;
- permettre l'inscription, la connexion et la déconnexion des utilisateurs ;
- permettre à chaque utilisateur connecté d'ajouter, modifier et retirer des livres de sa bibliothèque ;
- associer les livres à des catégories littéraires ;
- indiquer un statut de lecture : à lire, en cours ou terminé ;
- administrer les livres, catégories et bibliothèques depuis Django Admin ;
- prendre en charge les couvertures de livres, les favoris, la liste à lire plus tard, les notes, commentaires et statistiques.

1.4 Conformité au cahier des charges

Exigence du cahier des charges	Réalisation dans BookNest
Catalogue de livres	Vue BookListView, template book_list.html, recherche et filtre par catégorie
Fiche détaillée d'un ouvrage	Vue BookDetailView, template book_detail.html, résumé, catégorie, avis et note moyenne

Exigence du cahier des charges	Réalisation dans BookNest
Ajouter, modifier et supprimer un livre	Parcours CRUD via UserBookCreateView, UserBookUpdateView, UserBookDeleteView
Catégorie littéraire	Modèle Category, relation Book.category, page des catégories et administration
Statut de lecture	Champ reading_status du modèle UserBook : à lire, en cours, terminé
Django Admin	Modèles Category, Book et UserBook administrables
Inscription, connexion, déconnexion	Application accounts, SignUpView, LoginView, LogoutView
Image de couverture	Champ cover_image, configuration MEDIA_URL / MEDIA_ROOT, upload via formulaire
Bibliothèque par utilisateur	Modèle UserBook, relation entre User et Book, pages privées protégées
Favoris et à lire plus tard	Champs is_favorite, is_read_later, vues et pages dédiées
Note personnelle et commentaire	Champs personal_note, comment, rating, affichage des avis publics
Statistiques	Vue ReadingStatsView, répartitions par statut et catégorie, taux de complétion
Tests automatisés	48 tests Django exécutables avec python manage.py test
Documentation et GitHub	Rapport, README, requirements.txt, dépôt GitHub indiqué en page de garde

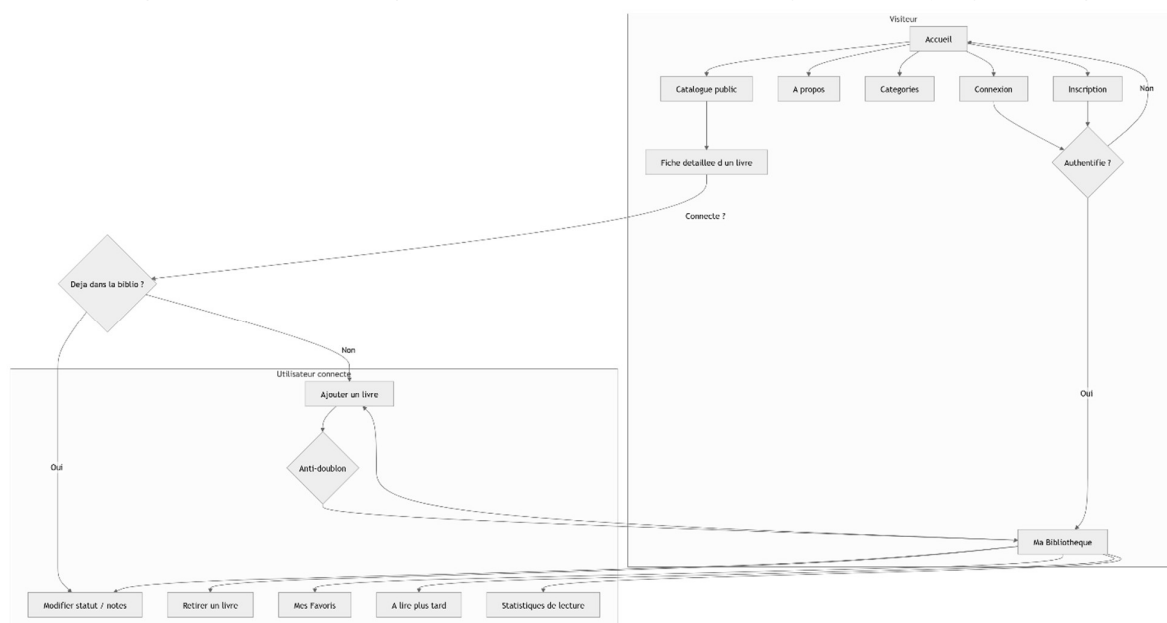
2. Analyse et conception

2.1 Acteurs

Acteur	Rôle
Visiteur	Consulte les pages publiques : accueil, à propos, catalogue, catégories et fiches livres. Il peut créer un compte.
Utilisateur connecté	Gère sa bibliothèque personnelle, ajoute des livres, modifie son statut de lecture, ajoute des notes, marque des favoris et consulte ses statistiques.
Administrateur	Gère les livres, catégories et relations utilisateur depuis l'interface Django Admin.

2.2 Cas d'utilisation principaux

Les cas d'utilisation sont organisés autour de trois acteurs : le Visiteur (navigation publique et création de compte), l'Utilisateur connecté (gestion de sa bibliothèque, statuts, favoris, notes et statistiques) et l'Administrateur (administration des livres et catégories via Django Admin).



2.3 Modèle de données

Le projet utilise trois entités principales : Category, Book et UserBook.

Category

Champ	Type	Rôle
name	CharField(100)	Nom unique de la catégorie
description	TextField	Description optionnelle

Book

Le modèle Book représente le catalogue commun. Un livre n'est pas créé plusieurs fois pour chaque utilisateur : il est stocké une fois, puis référencé par les bibliothèques personnelles.

Champ	Type	Rôle
title	CharField(250)	Titre du livre
author	CharField(250)	Auteur
isbn	CharField(13)	ISBN unique, optionnel
summary	TextField	Résumé du livre
publication_year	IntegerField	Année de publication
cover_image	ImageField	Image de couverture uploadable
category	ForeignKey(Category)	Catégorie littéraire
added_to_catalog	DateTimeField	Date d'ajout au catalogue

La contrainte `unique_together = ('title', 'author')` limite les doublons lorsque l'ISBN n'est pas renseigné.

UserBook

Le modèle UserBook relie un utilisateur à un livre du catalogue et stocke toutes les informations personnelles liées à la lecture.

Champ	Type	Rôle
user	ForeignKey(User)	Propriétaire de l'entrée
book	ForeignKey(Book)	Livre référencé dans le catalogue
reading_status	CharField	Statut : à lire, en cours, terminé
added_date	DateTimeField	Date d'ajout dans la bibliothèque
is_favorite	BooleanField	Marqueur favori
is_read_later	BooleanField	Marqueur à lire plus tard
personal_note	TextField	Note privée de l'utilisateur
comment	TextField	Commentaire public affiché sur la fiche livre
rating	IntegerField	Évaluation de 1 à 5
updated_at	DateTimeField	Date de dernière modification

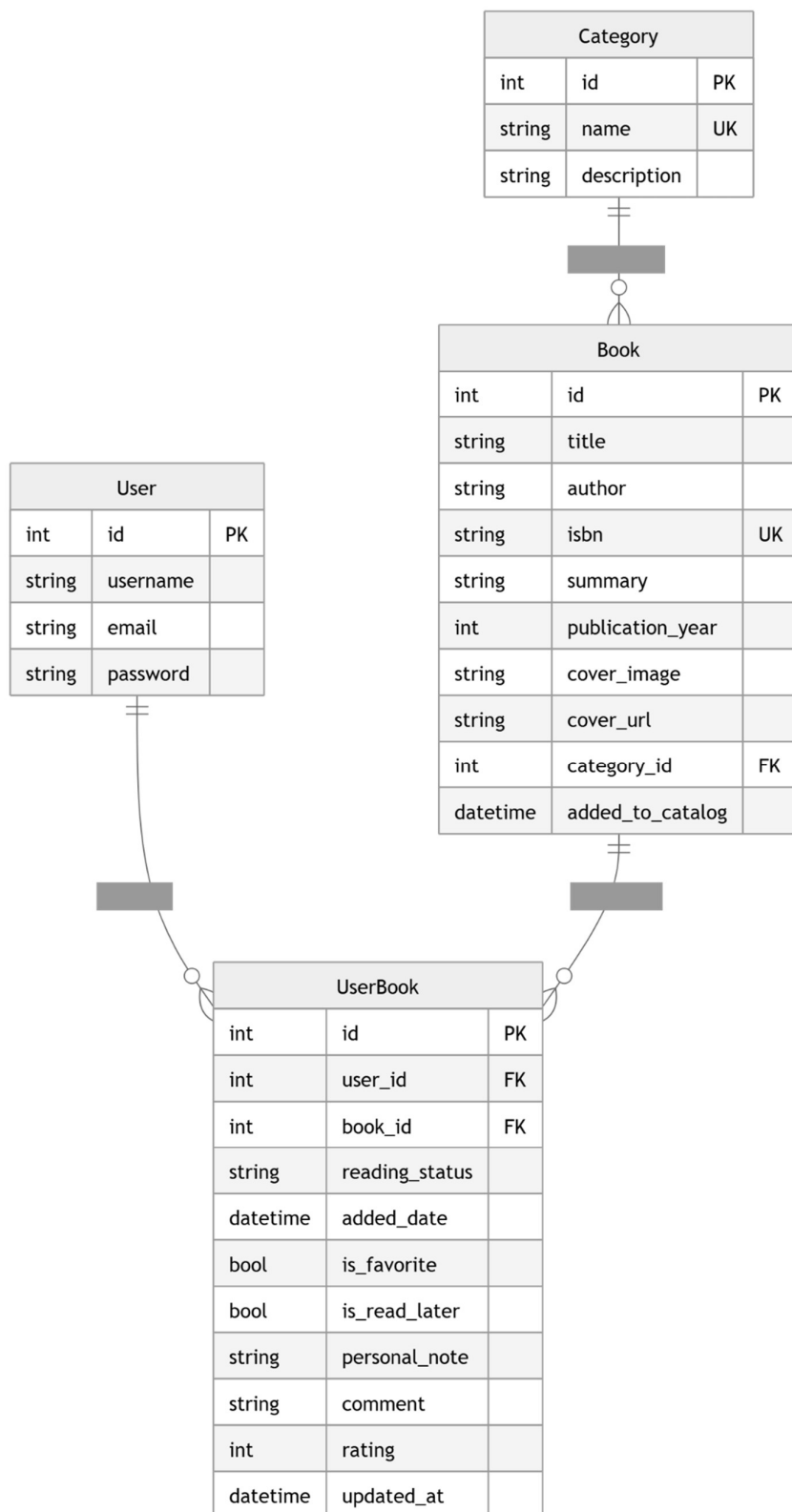
La contrainte `unique_together = ('user', 'book')` empêche un utilisateur d'ajouter deux fois le même livre dans sa bibliothèque.

2.4 Schéma relationnel

Le schéma relationnel s'articule autour de trois entités :

- Category → Book (un à plusieurs) : une catégorie classé plusieurs livres

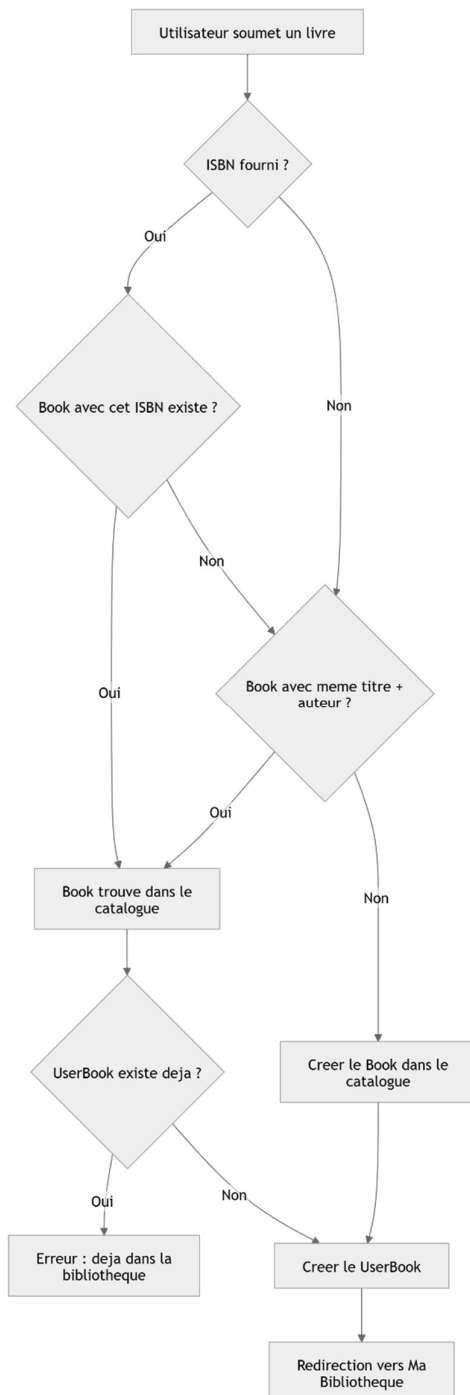
- Book → UserBook (un à plusieurs) : un livre peut être référencé dans plusieurs bibliothèques
- User → UserBook (un à plusieurs) : un utilisateur possède plusieurs entrées de bibliothèque



2.5 Logique anti-doublon

Lors de l'ajout d'un livre, la vue applique la logique suivante :

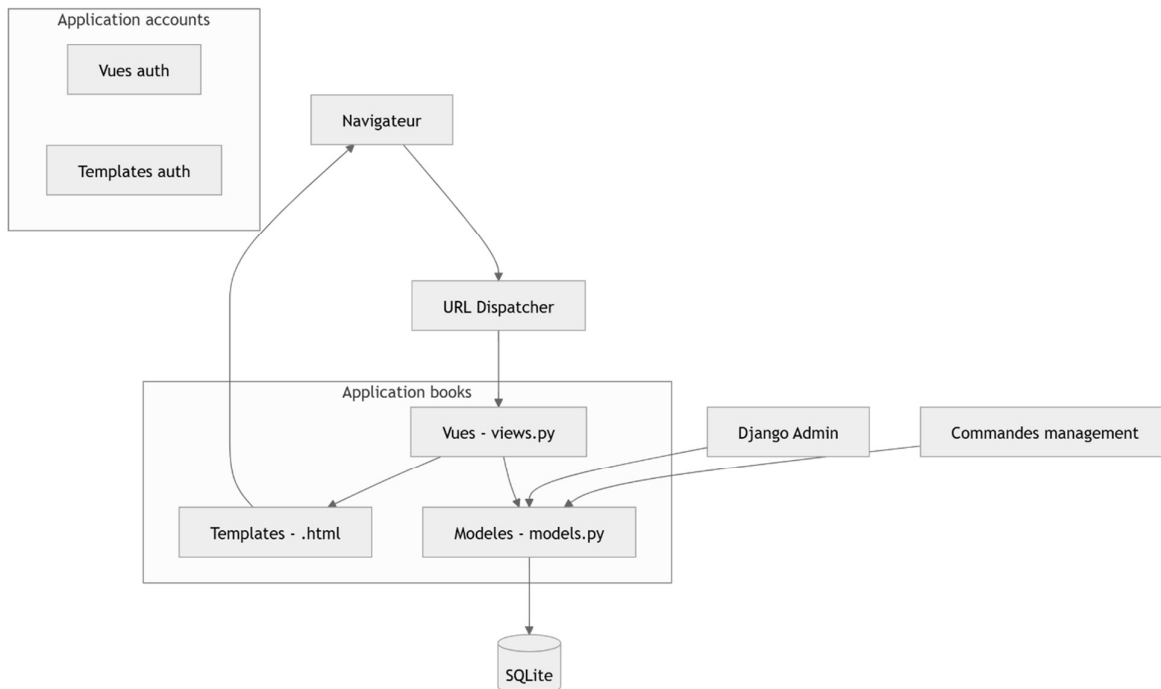
- Si un ISBN est fourni, recherche d'abord par ISBN
- Sinon, ou si non trouvé, recherche par titre et auteur (insensible à la casse)
- Si le livre n'existe pas encore dans le catalogue, il est créé
- Vérification que l'utilisateur n'a pas déjà ce livre dans sa bibliothèque
- Création du UserBook



3. Réalisation technique

3.1 Architecture générale

Le projet suit l'architecture MVT de Django : les modèles définissent les données, les vues traitent les requêtes, les templates affichent les pages HTML, et les URLs relient les routes aux vues. Django Admin permet l'administration des données.



Arborescence simplifiée

```

BookNest/
|-- booknest/           # Configuration du projet
|   |-- settings.py     # Apps, base SQLite, static, media, auth
|   |-- urls.py         # Routes principales
|-- books/              # Application principale
|   |-- models.py       # Category, Book, UserBook
|   |-- views.py        # Pages publiques, CRUD, favoris, stats
|   |-- urls.py         # Routes de l'application books
|   |-- admin.py        # Administration des modeles
|   |-- tests.py        # Tests automatisés
|-- accounts/           # Authentification
|   |-- views.py        # SignUpView
|   |-- urls.py         # signup, login, logout
|-- templates/          # Templates HTML
|-- static/css/         # Mise en forme
|-- media/book_covers/  # Couvertures uploadables
|-- requirements.txt
|-- manage.py
  
```

3.2 Applications et configuration

Application	Rôle
books	Catalogue, catégories, bibliothèque personnelle, CRUD, favoris, commentaires et statistiques
accounts	Inscription et intégration des vues Django de connexion/déconnexion

Extrait de configuration

```
LANGUAGE_CODE = 'fr'
TIME_ZONE = 'Africa/Casablanca'
STATIC_URL = 'static/'
MEDIA_URL = 'media/'
MEDIA_ROOT = BASE_DIR / 'media'
LOGIN_URL = '/accounts/login/'
LOGIN_REDIRECT_URL = '/my-books/'
LOGOUT_REDIRECT_URL = '/'
```

3.3 Modèles, migrations et administration

Les modèles `Category`, `Book` et `UserBook` ont été créés dans l'application `books`. Ils ont fait l'objet de migrations Django afin de créer la structure de la base SQLite. Le modèle `Book` inclut l'image de couverture via `ImageField`. Les trois modèles sont administrables depuis Django Admin.

3.4 URLs et vues

Routes publiques

Nom	URL	Vue	Rôle
home	/	HomeView	Accueil et statistiques globales
about	/about/	AboutView	Présentation de l'application
catalogue	/catalogue/	BookListView	Liste des livres, recherche et filtre
book-detail	/book/<pk>/	BookDetailView	Fiche détaillée, avis et note moyenne
category-list	/categories/	CategoryListView	Liste des catégories

Routes privées

Nom	URL	Vue	Rôle
my-books	/my-books/	UserBookListView	Bibliothèque personnelle
userbook-create	/my-books/add/	UserBookCreateView	Ajout d'un livre
userbook-update	/my-books/<pk>/edit/	UserBookUpdateView	Modification
userbook-delete	/my-books/<pk>/delete/	UserBookDeleteView	Retrait d'un livre
my-favorites	/my-books/favorites/	UserBookFavoriteListView	Livres favoris
my-read-later	/my-books/read-later/	UserBookReadLaterListView	Liste à lire plus tard

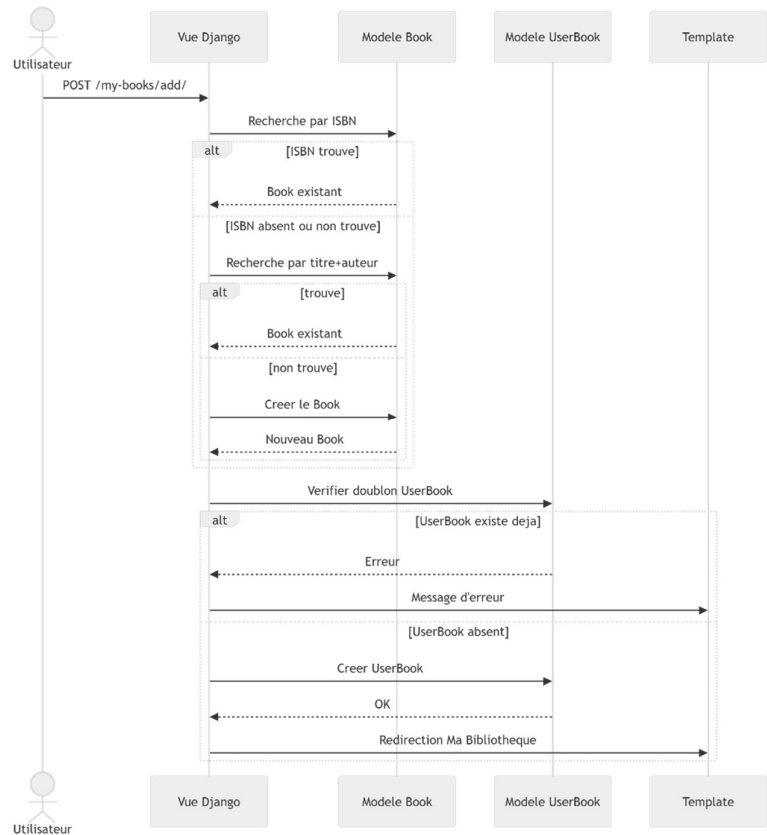
Nom	URL	Vue	Rôle
reading-stats	/my-books/stats/	ReadingStatsView	Statistiques de lecture

3.5 Templates et interface utilisateur

Template	Rôle
books/home.html	Page d'accueil
books/book_list.html	Catalogue, filtres et pagination
books/book_detail.html	Fiche détaillée avec couverture, note moyenne et commentaires
books/userbook_list.html	Bibliothèque, favoris et à lire plus tard
books/userbook_form.html	Ajout et modification d'un livre personnel
books/reading_stats.html	Statistiques de lecture
accounts/login.html	Connexion
accounts/signup.html	Inscription

3.6 Formulaires, CRUD et anti-doublon

Lors de l’ajout, la vue `UserBookCreateView` applique la logique anti-doublon, crée le livre si nécessaire, puis crée la relation `UserBook`. La modification met à jour le statut, le favori, la note et le commentaire. La suppression retire uniquement le livre de la bibliothèque personnelle sans toucher au catalogue commun.



3.7 Gestion des couvertures de livres

Le modèle Book prend en charge deux sources de couverture :

- Upload local : champ `cover_image` (ImageField), stocké dans `media/book_covers/`
- URL externe : champ `cover_url` (URLField), affiché comme image distante

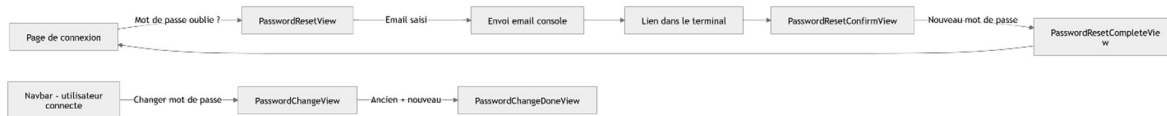
Dans les templates, l'upload local est prioritaire sur l'URL. Si aucun des deux n'est fourni, un placeholder « Sans couverture » est affiché.

3.8 Gestion des mots de passe

Le projet intègre les vues Django de gestion de mot de passe :

- Changement de mot de passe : `/accounts/password-change/` (accessible depuis la navbar, utilisateur connecté)
- Réinitialisation par email : `/accounts/password-reset/` (accessible depuis la page de connexion)

L'email de réinitialisation utilise le backend console (`django.core.mail.backends.console.EmailBackend`) : le lien de réinitialisation s'affiche dans le terminal de développement.

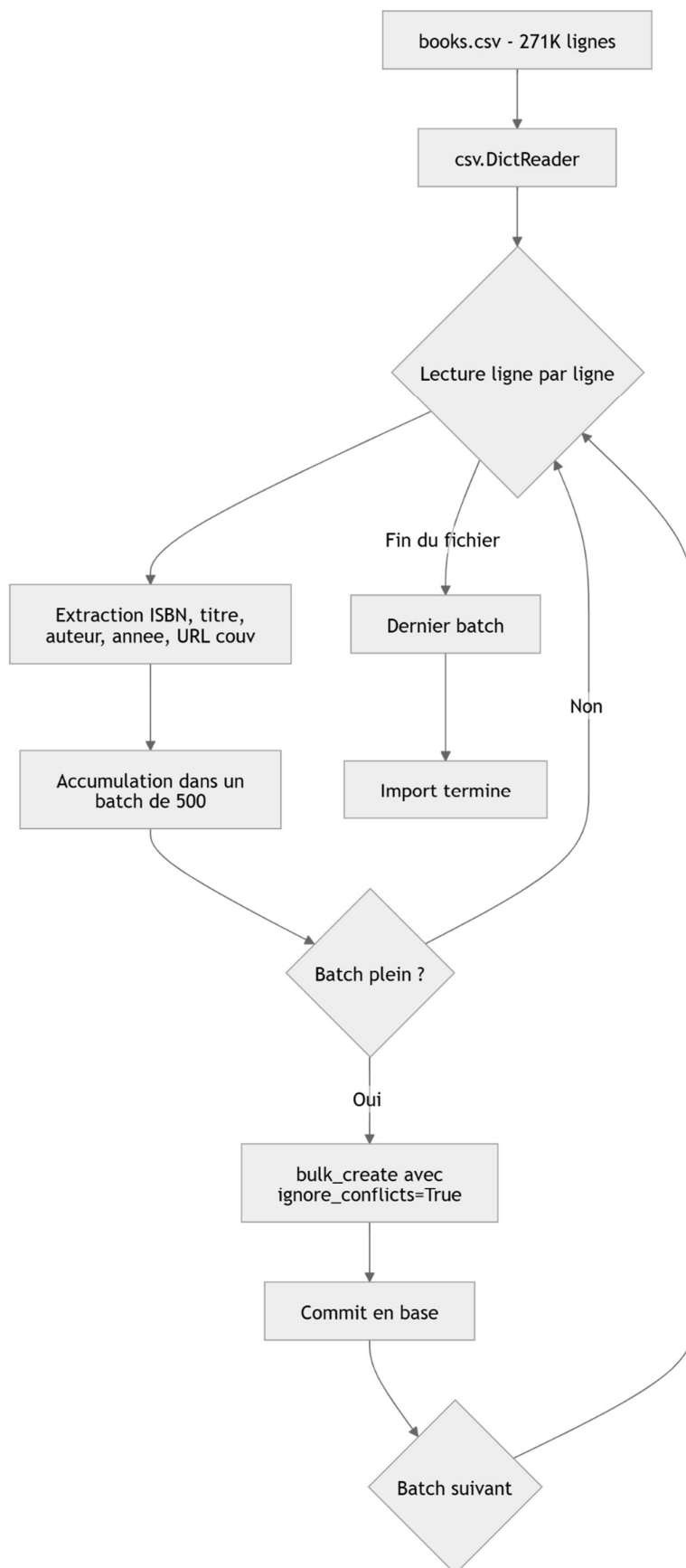


3.9 Import du catalogue (271 379 livres)

Une commande Django personnalisée (`python manage.py import_books`) a été développée pour importer le fichier `books.csv` contenant 271 379 références :

- Lecture ligne par ligne avec `csv.DictReader` (séparateur point-virgule)
- Gestion d'encodage UTF-8 avec remplacement des caractères invalides
- Traitement par lots de 500 livres avec `bulk_create(ignore_conflicts=True)`
- Récupération de l'URL de couverture (Image-URL-L ou Image-URL-M)

L'import complet s'exécute en environ 2 minutes. La commande est idempotente : les exécutions multiples n'entraînent pas de doublons.



3.10 Interface utilisateur et navigation

La barre de navigation a été réorganisée pour plus de clarté :

- Section publique à gauche : Accueil, Catalogue, Catégories, À propos
- Menu utilisateur regroupé dans un dropdown « Mon Espace » (CSS uniquement, sans JavaScript)
- Le dropdown contient : Ma Bibliothèque, Favoris, À lire plus tard, Statistiques, un séparateur, Changer mot de passe et Déconnexion
- Badge utilisateur affiché à côté du dropdown

3.11 Authentification et sécurité

- UserCreationForm pour l'inscription
- LoginView et LogoutView Django intégrés
- LoginRequiredMixin pour protéger les pages privées
- Filtrage du queryset avec `user=self.request.user` pour isoler les données
- Protection CSRF sur tous les formulaires

3.8 Suivi personnel, avis et statistiques

La note moyenne est calculée avec l'ORM Django :

```
ratings = UserBook.objects.filter(book=self.object).exclude(rating__isnull=True)
context['avg_rating'] = ratings.aggregate(avg=Avg('rating'))['avg']
context['rating_count'] = ratings.count()
```

La page de statistiques affiche : le nombre total de livres, la répartition par statut, la répartition par catégorie, les livres favoris, les livres les mieux notés, les derniers ajouts et le taux de complétion.

4. Tests et validation

4.1 Stratégie de test

Les tests automatisés sont écrits avec `django.test.TestCase` et couvrent les modèles, les pages publiques, l'authentification, les opérations CRUD, l'isolation des données entre utilisateurs, les favoris, la liste à lire plus tard, les statistiques et les évaluations.

Commande d'exécution

```
| python manage.py test
```

4.2 Résultat d'exécution

```
Found 48 test(s).
System check identified no issues (0 silenced).
.....
-----
Ran 48 tests in 58.051s

OK
```

4.3 Couverture fonctionnelle

Catégorie	Nombre	Éléments vérifiés
Modèles Category, Book, UserBook	15	Création, chaînes, contraintes d'unicité, relations et suppressions
Pages publiques	8	Accueil, à propos, catalogue, filtres, recherche, détail, catégories, 404
Authentification	7	Inscription, connexion, déconnexion, erreurs de formulaire
CRUD utilisateur	10	Accès protégé, création, modification, suppression, doublons, isolation
Fonctionnalités de suivi	8	Favoris, à lire plus tard, statistiques, notes moyennes, couvertures

4.4 Erreurs rencontrées et corrections

Erreur	Cause	Correction
ModuleNotFoundError: No module named 'books'	Application référencée avant création	Création des apps puis déclaration dans <code>INSTALLED_APPS</code>
Cannot use ImageField (Pillow)	Dépendance manquante	Installation de Pillow et ajout dans <code>requirements.txt</code>
NoReverseMatch sur routes	Incohérence entre noms namespacés	Harmonisation des noms d'URLs (<code>accounts:login</code> , etc.)
TemplateSyntaxError	Balises Django coupées par formatage HTML	Correction des templates + désactivation du formateur auto

4.5 Points de contrôle avant remise

Point de contrôle	Statut
python manage.py runserver démarre l'application	À vérifier avant soutenance
Migrations appliquées	Conforme au projet
Superutilisateur et Django Admin disponibles	Conforme au projet
CRUD fonctionnel	Vérifié par tests
python manage.py test réussit	48 tests OK
Dépôt GitHub accessible	Lien indiqué en page de garde
README, requirements.txt et .gitignore présents	Présents dans le dépôt
Rapport PDF et imprimé avec captures	À finaliser lors de l'export

5. Conclusion

5.1 Bilan

BookNest répond au cahier des charges du projet Django. L'application propose les pages publiques attendues, une gestion CRUD, des catégories, une authentification complète, une administration Django et une bibliothèque personnelle pour chaque utilisateur.

La réalisation va au-delà du socle minimal en intégrant directement dans les parcours principaux les couvertures de livres, les favoris, la liste à lire plus tard, les notes, les commentaires, les évaluations et les statistiques de lecture.

5.2 Compétences mises en pratique

- Création et configuration d'un projet Django
- Structuration en applications
- Modélisation des données et relations avec l'ORM
- Création de migrations
- Administration via Django Admin
- Vues génériques et URL dispatcher
- Templates avec héritage
- Formulaire POST et protection CSRF
- Authentification et protection des vues privées
- Gestion des fichiers media
- Tests automatisés Django
- Utilisation de Git et GitHub

5.3 Limites

- L'interface reste simple et pourrait être enrichie avec un framework CSS
- La recherche pourrait devenir plus avancée avec une recherche plein texte
- Les commentaires ne disposent pas encore d'un système de modération
- Il n'existe pas encore d'import/export de bibliothèque
- L'application n'intègre pas encore de recommandations de lecture

5.4 Pistes d'évolution

- Ajouter une API REST avec Django REST Framework
- Permettre l'import/export CSV ou JSON
- Mettre en place une recherche plein texte
- Ajouter un système de modération des commentaires
- Proposer des recommandations basées sur les catégories ou les notes
- Préparer un déploiement en ligne avec une base PostgreSQL

6. Annexes

A. Instructions d'installation

```
# 1. Cloner le depot
git clone https://github.com/Tranck04/django-booknest-tracy-franck-mukendi.git
cd django-booknest-tracy-franck-mukendi

# 2. Creer et activer l'environnement virtuel
python -m venv .venv
.venv\Scripts\activate

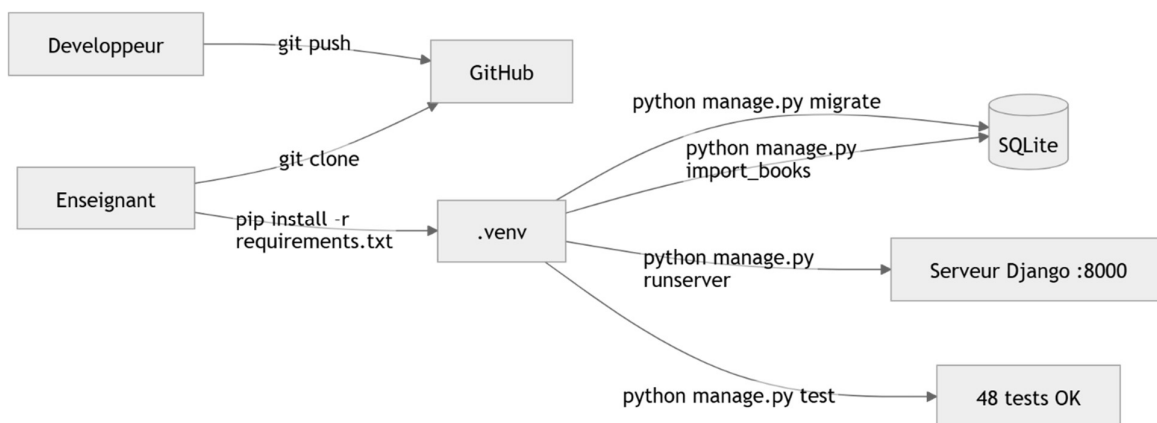
# 3. Installer les dependances
pip install -r requirements.txt

# 4. Appliquer les migrations
python manage.py migrate

# 5. Creer un superutilisateur
python manage.py createsuperuser

# 6. Lancer le serveur
python manage.py runserver

# 7. Executer les tests
python manage.py test
```



B. Dépendances principales

```
Django==6.0.6
pillow==12.2.0
```

C. Dépôt GitHub

Dépôt du projet : <https://github.com/Tranck04/django-booknest-tracy-franck-mukendi>